

Proofs for Planar Triangulation Algorithm

1 Introduction

The algorithm follows a tree-like structure for generating all valid planar triangulations of a biconnected planar graph. To prove the correctness and efficiency of our algorithm, we must sequentially prove different elements: completeness, existence of valid root triangulations, and complexity analysis.

2 Proof of Completeness

2.1 Pruning Strategy

Our algorithm employs pruning operations for triangulations where a blocking chord is a duplicate chord (i.e., the chord has occurred in earlier faces). When we encounter such duplication, we prune that triangulation and its entire branch.

Theorem 1 (Completeness via Pruning). *All children of a pruned branch would have been invalid triangulations.*

Proof. We classify triangulations as invalid when any one of their blocking chords is a duplicate chord. To prove completeness, we must demonstrate that a blocking chord is never flipped in the children of a branch.

Let chord $a-b$ be a blocking chord in any triangulation tree node. Since the node represents a triangulation, every face is triangulated. Therefore, there exist two other points that form triangles with the $a-b$ chord on both sides:

- Let c be the point on the opposite side of the root vertex
- Let r be the point on the root side

We analyze three cases based on the position of r :

Case 1: r is on the left of the root vertex

Flipping the $a-b$ chord would result in an $r-c$ chord. However, before flipping, the $b-r$ chord was also a blocking chord positioned to the left of $a-b$. This means $a-b$ could not be the leftmost blocking chord. Consequently, the new $r-c$ chord cannot be the leftmost blocking chord either, because the $r-b$ chord still exists and remains to the left of the $c-r$ chord. Therefore, flipping the $a-b$ blocking chord is not permitted by the algorithm in this case.

Case 2: r is on the right of the root vertex

Flipping the $a-b$ chord would produce an $r-c$ chord. However, the $a-r$ chord is more to the left than the $r-c$ chord, making this flip invalid according to the algorithm.

Case 3: r is at the root

Flipping the a - b chord would create a root- c chord. This would give the child triangulation's root vertex better visibility than the parent, which violates the algorithm's constraints.

In all three cases, flipping a blocking chord in a triangulation tree is impossible. Therefore, a blocking chord will be present in all of its children.

Thus, if a blocking chord is a duplicate chord, all child triangulations will contain this chord, making all child triangulations invalid. By pruning them, we do not ignore any valid triangulations. $\square$

2.2 Finding Valid Root Triangulations

Having proved that our pruning method preserves all valid triangulations, we now address whether we can always find a root triangulation where no chord is duplicate or blocking.

3 Existence of Valid Root Triangulation

Definition 1 (Root Triangulation). A root triangulation is the root of the tree of triangulations, constructed by selecting a root vertex and adding chords from this root vertex to all non-neighboring vertices of the current face.

Definition 2 (Safe Root Vertex). A safe root vertex is one from which adding chords to every other non-neighboring node will not create a multi-edge problem. Formally, we must find a vertex such that none of the previous faces share a chord where one endpoint is the root vertex and the other endpoint is a vertex of the current face.

Theorem 2 (Existence of Safe Root). *In any face of a biconnected planar graph, there always exist at least two safe vertices that can serve as root vertices for valid triangulations.*

Proof. Consider a face with n vertices, numbered from 1 to n . Since we are working with a planar graph, chords can only exist in the outer plane.

Case 1: No chords from previous faces

If there are no chords from previous faces with both endpoints inside the current face, then all n vertices are eligible to be safe vertices.

Case 2: At least one chord from previous faces

Suppose there exists a chord a - b on the outer plane belonging to a previous face. Both a and b are disqualified from being safe vertices. However, this introduces an important constraint.

Due to planarity, chords cannot cross each other. Since the a - b chord exists, no chord on the outer plane can overtake it. Consequently:

- Nodes $(a + 1, a + 2, \dots)$ might have connections between them (if not neighbors)
- They cannot have chords with nodes $(a - 1, \dots, b + 1)$ as this would violate planarity by crossing the a - b chord

This creates a specific structure we denote as T_n , where n is the number of internal vertices between the endpoints of a constraining chord.

3.1 Analysis of T_n Structures

The current face contains two complementary T_n structures:

1. Structure 1: $a, a + 1, \dots, b$
2. Structure 2: $a, a - 1, \dots, b$

The internal vertices are numbered $1, \dots, n - 3$. The minimum structure is T_1 (since T_0 would mean a and b are neighbors, and neighbors cannot form chords). The maximum is T_{n-3} (accounting for endpoints and the complementary structure).

Base Case: T_1

For T_1 , there is one internal vertex c between a and b . Both a and b are neighbors of c , making any chord from c impossible. Therefore, c is a safe vertex.

Base Case: T_2

For T_2 , with vertices c and d between a and b :

- If c has a chord, it must be with b (the only non-neighbor)
- This creates a T_1 structure containing d , making d safe
- Similarly, if d has a chord, it must be with a
- This creates a T_1 structure containing c , making c safe

General Case: T_n for arbitrary n

We present an algorithm to find a safe vertex in $O(n)$ time:

Algorithm 1 Finding a Safe Vertex in T_n

```

1: Consider vertex  $a + 1$ 
2:  $a + 1$  cannot create chords with neighbors  $a$  and  $a + 2$ 
3: Possible chord partners:  $\{a + 3, \dots, b\}$ 
4: Check from  $b$  downward using a hashset ( $O(1)$  per check)
5: if chord  $(a + 1, b)$  exists then
6:   Structure becomes  $T_{n-1}$  with vertices  $a + 2, \dots, b - 1$ 
7: else if chord  $(a + 1, b - k)$  exists for some  $k > 0$  then
8:   Structure becomes  $T_{n-k-1}$ 
9: else
10:    $a + 1$  is a safe vertex (no chords with any candidate)
11: end if

```

In each iteration:

- If a chord is found, n decreases, eventually reaching T_1 or T_2
- If no chord is found after checking all candidates, the vertex is safe

Since every check reduces the problem size or finds a safe vertex, and we've proven T_1 and T_2 contain safe vertices, every T_n structure contains at least one safe vertex.

Conclusion:

From a face with no multi-edges in previous faces, all n vertices can be safe. For faces with at least one multi-edge from previous faces, there are at least two complementary T_n structures, each containing at least one safe vertex.

Therefore, every face in a biconnected planar graph, regardless of calculation stage, has at least two safe vertices. This guarantees the existence of at least two safe root triangulations regardless of the calculation order. $\square$

4 Completeness Summary

We have now established:

1. The pruning method preserves all valid triangulations
2. Safe root triangulations always exist

Therefore, our algorithm covers all valid planar triangulations of any planar biconnected graph.

5 Time Complexity Analysis

Theorem 3 (Amortized Linear Time Complexity). *The algorithm has amortized $O(1)$ time complexity per triangulation, or amortized linear time complexity overall.*

5.1 Building Time

Definition 3 (Building Time). The building time is the time required to calculate the safe root and construct the chords from the root vertex for a single face.

From our algorithm and proofs:

- Finding a safe vertex: $O(n)$
- Adding all chords from the root: $O(n)$
- Total building time for a face with n vertices: $O(n)$

5.2 Minimum Number of Triangulations

Lemma 4 (Lower Bound on Triangulation Count). *For a face with n vertices, there are at least $O(n)$ valid triangulations.*

Proof. We have proven that every face has at least two safe vertices. Let vertices a and c be two such safe vertices.

By our algorithm, starting with the a -root triangulation, we can reach the c -root triangulation through a series of chord flips. At each step, we flip exactly one chord, and each intermediate triangulation is valid (otherwise it would have been pruned).

Comparing a -root and c -root triangulations:

- a -root: all chords have endpoint a
- c -root: all chords have endpoint c
- Only one chord may be shared: $a-c$
- All other $n - 4$ chords must be different

Therefore, transforming from a -root to c -root requires $n - 4$ flips, each producing a valid triangulation.

Including the root triangulation: at least $n - 4 + 1 = n - 3$ triangulations exist.

Thus, there are at least $O(n)$ triangulations for a face with n vertices. $\square$

5.3 Amortized Analysis

Consider the recursive tree structure of our algorithm for a biconnected graph with multiple faces.

Two-Face Graph:

Let n_1 and n_2 be the number of vertices in faces 1 and 2, respectively.

$$\text{Total building time: } B = n_1 + n_1 \cdot n_2$$

$$\text{Total triangulations: } T = n_1 \cdot n_2$$

Since $n_2 \geq 3$:

$$\begin{aligned} n_1 \cdot 1 &< n_1 \cdot n_2 \\ B = n_1 + n_1 \cdot n_2 &< n_1 \cdot n_2 + n_1 \cdot n_2 \\ &= 2 \cdot n_1 \cdot n_2 \\ &= 2T \end{aligned}$$

Total time complexity: $< 3T$

Three-Face Graph:

Let n_1, n_2, n_3 be the number of vertices in the three faces.

$$\begin{aligned} B &= n_1 + n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3 \\ T &= n_1 \cdot n_2 \cdot n_3 \end{aligned}$$

From the two-face case: $n_1 + n_1 \cdot n_2 < 2 \cdot n_1 \cdot n_2$

Therefore:

$$B < 2 \cdot n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3$$

Since $n_3 \geq 3$, we have $2 < n_3$:

$$\begin{aligned} 2 \cdot n_1 \cdot n_2 &< n_1 \cdot n_2 \cdot n_3 \\ B &< n_1 \cdot n_2 \cdot n_3 + n_1 \cdot n_2 \cdot n_3 \\ &= 2T \end{aligned}$$

Total time complexity: $< 3T$

General Case: k Faces

By induction, for any number of faces where each has at least 3 vertices:

$$\begin{aligned} B &= n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_k \\ T &= n_1 \cdot n_2 \cdots n_k \end{aligned}$$

The pattern holds:

$$n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_{k-1} < 2 \cdot n_1 \cdot n_2 \cdots n_{k-1}$$

Since $n_k \geq 3$:

$$\begin{aligned} 2 \cdot n_1 \cdot n_2 \cdots n_{k-1} &< n_1 \cdot n_2 \cdots n_k \\ B &< 2T \end{aligned}$$

Total time complexity: $< 3T$

Corollary 5. *For any biconnected planar graph, the algorithm takes total time $< 3 \times$ (number of triangulations). Therefore, the algorithm has amortized linear time complexity.*

6 Space Complexity Analysis

Theorem 6 (Linear Space Complexity). *The algorithm has $O(n)$ space complexity, where n is the total number of vertices.*

Proof. When processing face i , we must store information for all faces $1, \dots, i$.

The storage includes:

- Vertex information: $n_1 + n_2 + \dots + n_i$ vertices
- Chord information: For a planar graph, there are $n - 3$ chords total
- A hashset for current chord lookup

Since the total number of vertices across all faces is $O(n)$ and the number of chords in a planar graph is also $O(n)$, the total space complexity is $O(n)$.

The hashset for storing current chord information also requires $O(n)$ space.

Therefore, the entire algorithm uses $O(n)$ space complexity. $\square$

7 Conclusion

We have rigorously proven that our planar triangulation algorithm:

1. **Is complete:** It finds all valid planar triangulations through proper pruning of invalid branches
2. **Always has a starting point:** At least two safe root vertices exist in every face
3. **Is efficient in time:** Amortized $O(1)$ time complexity per triangulation, or amortized linear overall
4. **Is efficient in space:** $O(n)$ space complexity

These properties make the algorithm both theoretically sound and practically efficient for enumerating all planar triangulations of biconnected planar graphs.